CAHIER DES CHARGES

Application web de gestion de flotte de bus

Bestimetravel — Gestion des Bus

Suivi des véhicules, des contrats, du carburant et de la rentabilité, sur téléphone comme sur ordinateur

Projet	Bestimetravel · Gestion des Bus (application web progressive, PWA)
Version	1.0 — Novembre 2026
Réalisé par	Bilel
Application en ligne	https://bus-app-web-lemon.vercel.app
Code source	https://github.com/bilelsdn21/bus-app-web
Technologies	React · TailwindCSS · FastAPI (Python) · PostgreSQL (Supabase) · Vercel · Render

Sommaire

1. Présentation du projet
2. Expression des besoins
3. Conception et architecture technique
4. Interface et ergonomie (mobile d'abord)
5. Qualité, tests et recette
6. Planning et organisation
7. Budget et hébergement
8. Risques et limites connues
9. Évolutions prévues
10. Livrables et conditions de recette

A. Annexe — Glossaire
B. Annexe — Guide rapide utilisateur
C. Annexe — Configuration et installation
D. Annexe — Trame de soutenance et scénario de démonstration

1. Présentation du projet

> **L'essentiel en une phrase**
> **23 fonctions sur 25** livrées (2 prévues) · **3 rôles** d'accès · **10 exports** Excel · **24 tests** automatiques · calendrier chargé avec **8 requêtes au lieu de 52** · hébergement à **0 TND par mois** · installable sur téléphone (PWA).

1.1 Contexte

Bestimetravel exploite une flotte de bus, minibus et autocars (types **MICRO**, **OTOKAR** et **BUS**) répartie sur plusieurs régions de Tunisie, notamment **Sousse** et **Djerba**. Chaque véhicule est exploité dans le cadre d'un **contrat** assorti d'un loyer. Les **sorties** (excursions, transferts) que le véhicule effectue génèrent le chiffre d'affaires ; le carburant constitue la principale charge variable.

Jusqu'ici, ce suivi reposait sur un classeur Excel avec des macros VBA, puis sur un prototype Streamlit. L'application décrite dans ce document remplace ce fonctionnement par un véritable système d'information accessible en ligne.

1.2 Problématique

Limite de l'ancien fonctionnement	Conséquence
Rattachement d'une sortie à un véhicule par son **nom** (texte)	Une simple différence de nom a mal affecté des recettes : **1 440 TND perdus** lors d'un incident réel
Macros VBA fragiles (erreurs de compilation, sécurité des macros)	Fichier parfois inutilisable, dépendance à un poste unique
Aucun accès fiable depuis le terrain (téléphone)	Saisie différée, oublis, informations obsolètes
Carburant saisi par mois civil alors que les contrats ont des dates libres	Calcul manuel du résultat réel d'un contrat, source d'erreurs
Aucune traçabilité des modifications	Impossible de savoir qui a changé quoi, et quand
Véhicules qui changent de région en cours de mois	Chiffres attribués à la mauvaise région, totaux incohérents

1.3 Objectifs

N°	Objectif	Indicateur de réussite
O1	Fiabiliser tous les calculs (source unique de vérité, intégrité des données)	Aucune erreur d'affectation possible (clés étrangères) ; 24 tests automatiques au vert
O2	Rendre l'outil utilisable partout, surtout sur téléphone	Application installable, vue « cartes » mobile, saisie d'un jour en quelques touchers
O3	Donner la rentabilité par véhicule, contrat et région, en temps réel	Net, pourcentage et totaux régionaux affichés dès le chargement du calendrier
O4	Assurer la traçabilité et le contrôle des accès	Journal d'activité complet ; trois niveaux de droits
O5	Être rapide	Calendrier du mois affiché en environ 1 à 2 secondes en régime normal
O6	Faciliter la communication avec l'équipe	Notifications push (résumé du mois, annonces)

1.4 Périmètre

Inclus dans le projet	Hors périmètre (évolutions possibles)
Gestion des véhicules, contrats, carburant, destinations et tarifs Calendrier mensuel et saisie des sorties Régions datées et changements de région ou de contrat en cours de mois Véhicules loués en renfort et vérification d'alternative Analyse mensuelle (occupation, couverture du loyer) Exports Excel, journal d'audit, notifications push Application installable (PWA), vue mobile	Facturation client et émission de factures PDF Comptabilité générale, paie des chauffeurs Suivi GPS / télématique des véhicules Paiement en ligne, portail client Application native publiée sur les magasins d'applications Gestion des comptes utilisateurs depuis l'interface (les comptes sont aujourd'hui définis côté serveur)

1.5 Utilisateurs et rôles

Rôle	Qui	Droits
Propriétaire (owner)	Direction	Tous les droits de l'administrateur, plus la supervision : journal complet incluant les connexions et déconnexions
Administrateur (admin)	Gestionnaires, exploitation	Lecture et écriture : saisie des sorties, contrats, carburant, véhicules, tarifs, régions ; analyse ; exports ; envoi d'annonces ; journal des modifications
Lecture seule (viewer)	Équipe, consultation	Consultation du calendrier, des contrats, des véhicules loués et des paramètres. Aucune modification possible (refusée aussi côté serveur)

2. Expression des besoins

2.1 Besoins fonctionnels

Priorité selon la méthode MoSCoW (**Must** = indispensable, **Should** = important, **Could** = souhaitable). L'état correspond à la version 1.0 livrée.

Réf.	Fonction	Description	Priorité	État
F01	Authentification	Connexion par identifiant et mot de passe ; jeton signé avec expiration ; déconnexion	Must	Réalisé
F02	Rôles et droits	Trois rôles (propriétaire, administrateur, lecture seule) ; toute écriture est contrôlée côté serveur	Must	Réalisé
F03	Calendrier mensuel	Grille véhicules × jours avec pastilles de couleur, net du mois, pourcentage, regroupement par région, totaux	Must	Réalisé
F04	Vue « cartes » mobile	Une carte par véhicule avec mini-calendrier du mois ; bascule Cartes / Tableau ; ouverture sur aujourd'hui	Should	Réalisé
F05	Saisie d'un jour	Ajout, modification, suppression des sorties d'un véhicule pour un jour : catégorie, destination, heures, client, passagers, prix calculé automatiquement	Must	Réalisé
F06	Sorties sur plusieurs jours	Une excursion peut couvrir une période ; le jour apparaît « réservé »	Must	Réalisé

Réf.	Fonction	Description	Priorité	État
F07	Indisponibilités	Entretien, Libre, Repos chauffeur : le véhicule est marqué indisponible ou libre	Must	Réalisé
F08	Contrôle de couverture	Impossible de saisir une sortie un jour non couvert par un contrat	Must	Réalisé
F09	Contrats	Période libre (dates quelconques), loyer, libellé ; résultat du contrat (recette − loyer − carburant)	Must	Réalisé
F10	Carburant	Carburant mensuel par véhicule : estimé, puis réel ; réparti au prorata des jours du contrat	Must	Réalisé
F11	Véhicules	Création, modification, suppression protégée ; type, plaque, région, distance, ordre d'affichage	Must	Réalisé
F12	Régions datées	Historique de région par véhicule ; enregistrement d'un déplacement à une date sans modifier le passé	Must	Réalisé
F13	Changement en cours de mois	Un véhicule qui change de région ou de contrat au milieu du mois apparaît une fois par région, avec ses propres jours et son propre résultat (Calendrier et Analyse)	Must	Réalisé
F14	Destinations et tarifs	Catalogue de destinations avec prix par type de véhicule (MICRO, OTOKAR, BUS)	Must	Réalisé
F15	Périodes de prix	Prix historiques par période, appliqués automatiquement selon la date de la sortie	Should	Réalisé
F16	Véhicules loués en renfort	Location externe avec coût réparti au prorata sur les mois concernés ; total informatif sur le calendrier	Should	Réalisé
F17	Vérification d'alternative	Pour une location, indique jour par jour si un véhicule propre du même type était libre dans la région	Could	Réalisé
F18	Analyse mensuelle	Par véhicule : couverture du loyer, jours travaillés, taux d'occupation, heures d'activité ; même découpage que le calendrier en cas de changement de contrat ou de région	Should	Réalisé
F19	Exports Excel	Calendrier, contrats, véhicules, destinations, régions, prix, locations, analyse, journal	Must	Réalisé
F20	Journal d'activité	Qui a ajouté, modifié ou supprimé quoi, et quand ; connexions visibles du propriétaire uniquement	Should	Réalisé
F21	Notifications push	Abonnement par téléphone ; annonces envoyées par un administrateur ; résumé du mois	Should	Réalisé
F22	Application installable	PWA : icône sur l'écran d'accueil, plein écran, consultation hors ligne des dernières données	Should	Réalisé
F23	Paramètres	Seuils horaires matin / soir / nuit (13 h et 22 h par défaut), date d'application des prix actuels	Should	Réalisé
F24	Gestion des comptes	Création et modification des utilisateurs depuis l'interface	Could	Prévu
F25	Sauvegarde automatique	Sauvegarde planifiée de la base (un script manuel existe)	Should	Prévu

2.2 Règles de gestion

R1 — Résultat d'un contrat

Résultat = recettes − loyer − carburant. Les recettes sont la somme des sorties payantes dont la date tombe dans la période du contrat. Le pourcentage est le résultat divisé par le loyer.

R2 — Carburant réparti au prorata des jours

Le carburant est saisi par **mois civil**, alors que les contrats ont des dates libres. Chaque mois contribue au prorata du nombre de jours qu'il partage avec le contrat. On utilise le montant **réel** s'il existe, sinon l'**estimation** ; le résultat est alors marqué « estimé ».

> **Exemple chiffré (vérifié par un test automatique)**
> Contrat du 15 mai au 14 juin, loyer 5 000 TND. Carburant de mai (réel) 2 000 TND, de juin (estimé) 1 200 TND.
> Recettes : 12 270 TND.
> Carburant = 17/31 × 2 000 + 14/30 × 1 200 = 1 096,77 + 560,00 = **1 656,77 TND**
> Résultat = 12 270 − 5 000 − 1 656,77 = **5 613,23 TND (estimé)**

R3 — Présence d'un véhicule au calendrier

Un véhicule n'apparaît sur le calendrier d'un mois que si **au moins un contrat chevauche ce mois**. Les jours hors contrat sont grisés et ne peuvent pas recevoir de saisie.

R4 — Couleur d'un jour

La couleur dépend de la **période de la journée** touchée par les sorties, d'après leurs heures de début et de fin. Les seuils (13 h et 22 h par défaut) sont modifiables dans les paramètres.

Couleur	Signification	Condition
Jaune	Matin	Sorties uniquement avant 13 h
Orange	Soir	Sorties entre 13 h et 22 h
Violet	Nuit	Sorties après 22 h
Vert	Réservé	Excursion sur plusieurs jours, ou sorties couvrant plusieurs périodes de la journée
Rouge	Indisponible	Entretien ou repos chauffeur (sans sortie payante)

R5 — Recette affectée à la date exacte

Chaque sortie compte dans la région où se trouvait le véhicule **le jour de la sortie**. Rien n'est jamais déplacé rétroactivement.

R6 — Véhicule qui change de région dans un contrat

Le contrat est découpé en tronçons d'une seule région. La **recette** de chaque tronçon est exacte ; le **loyer** et le **carburant** sont répartis au prorata des jours (ligne marquée « part »). Les tronçons additionnés redonnent exactement le total du contrat, au centime près.

R7 — Véhicule qui change de contrat en cours de mois

Si les contrats du mois sont dans des régions différentes (par exemple contrat A à Sousse jusqu'au 12, contrat B à Djerba à partir du 13), le véhicule apparaît **une ligne par contrat, dans le bloc de sa région**. Chaque ligne a ses jours et son résultat ; les jours entre deux contrats restent grisés. Les totaux régionaux additionnent correctement tous les loyers.

R8 — Véhicule loué en renfort

Le coût total d'une location est réparti **au prorata des jours** sur chaque mois qu'elle traverse. Il est affiché à titre d'information et n'est pas déduit du net de la flotte.

R9 — Noms de région normalisés

« Djerba », « djerba » ou « Djerba » avec un espace en trop désignent la même région : les noms sont ramenés à une écriture unique à chaque lecture et à chaque enregistrement. Une région ne peut donc plus apparaître en double sur le calendrier.

2.3 Besoins non fonctionnels

Domaine	Exigence	Mise en œuvre / mesure
Performance	Calendrier du mois affiché en 1 à 2 s en régime normal	Chargement groupé : 52 requêtes SQL ramenées à 8 ; cache serveur ; réponses compressées ; cache navigateur avec affichage immédiat
Disponibilité	Application accessible 24 h / 24	Hébergement gratuit avec mise en veille après 15 min : atténuée par une surveillance toutes les 5 min ; relances automatiques côté application
Sécurité	Accès contrôlé, données protégées	Jeton signé et expirant, rôles contrôlés côté serveur, limitation des tentatives de connexion, HTTPS, en-têtes de sécurité, RLS sur la base (voir § 3.6)
Ergonomie mobile	Utilisable d'une main sur téléphone	Barre d'onglets en bas, cartes par véhicule, boutons principaux d'au moins 44 px, zoom autorisé, champs en 16 px (pas de zoom automatique sur iPhone)
Compatibilité	Navigateurs et téléphones récents	Chrome, Edge, Firefox, Safari récents ; Android ; iPhone (iOS 16.4 ou plus) pour les notifications, après ajout à l'écran d'accueil
Maintenabilité	Code lisible et testé	Architecture en couches (MVC), un seul module de calcul, 24 tests automatiques, documentation (README, guide de déploiement)
Traçabilité	Savoir qui a fait quoi	Journal d'activité de chaque ajout, modification, suppression, avec utilisateur et heure
Sobriété	Chargement léger sur réseau mobile	Environ 75 Ko de code (compressé) ; aucune bibliothèque lourde
Portabilité	Développement local simple	SQLite en local, PostgreSQL en production : même code, une variable d'environnement

3. Conception et architecture technique

3.1 Vue d'ensemble

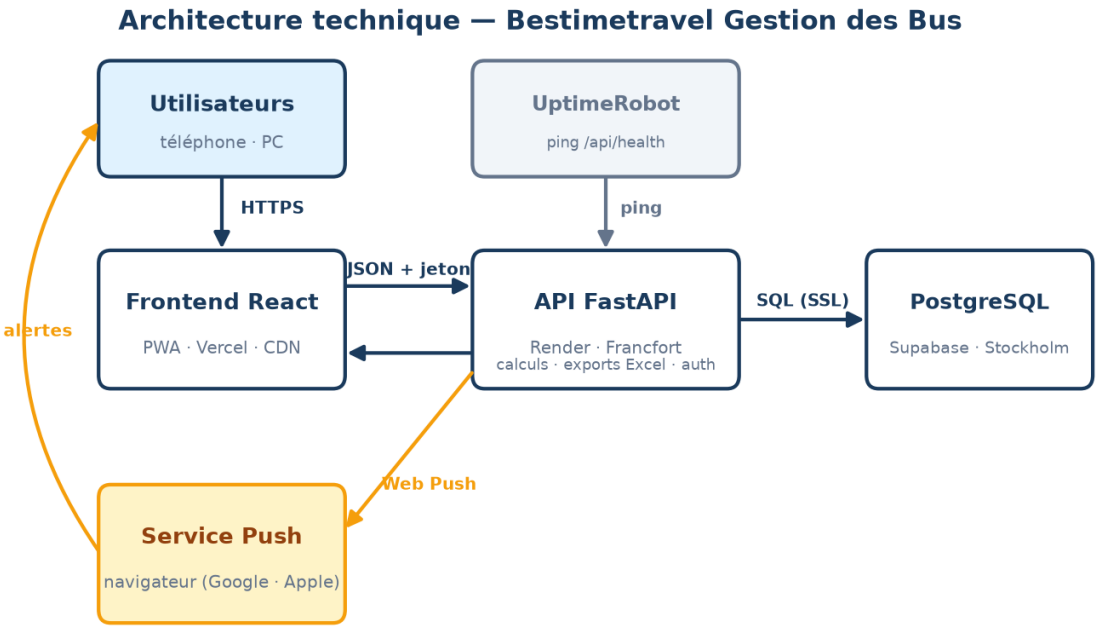


Figure 1 — Architecture technique

L'utilisateur accède à une application **React** servie par Vercel. Celle-ci dialogue en HTTPS avec une **API FastAPI** hébergée à Francfort, qui lit et écrit dans une base **PostgreSQL** (Supabase, Stockholm). Les notifications passent par le service Push du navigateur. Un service de surveillance interroge régulièrement l'API pour qu'elle reste éveillée.

3.2 Choix techniques

Couche	Technologie	Justification
Interface	React 18, Vite, TailwindCSS	Composants réutilisables, build rapide, interface responsive sans CSS lourd
API	FastAPI (Python), Pydantic	Validation automatique, documentation interactive (/docs), performances
Accès aux données	SQLAlchemy (ORM)	Requêtes paramétrées (pas d'injection SQL), mêmes modèles en SQLite et PostgreSQL
Base de données	PostgreSQL (Supabase)	Base relationnelle fiable, clés étrangères, hébergement gratuit
Hébergement	Vercel (interface), Render (API), Supabase (base)	Déploiement automatique à chaque envoi sur GitHub ; coût nul
Notifications	Web Push (VAPID), pywebpush	Standard ouvert, sans application native
Exports	openpyxl	Fichiers Excel natifs, fidèles à l'habitude des utilisateurs

3.3 Architecture de l'API (MVC)

Couche	Dossier	Rôle
Modèle	models.py	Tables et relations (SQLAlchemy)
Vue	schemas.py	Format des requêtes et réponses (Pydantic)
Contrôleurs	controllers/	Un routeur par ressource : fines couches HTTP
Services	services/	Logique métier : calc.py (calculs purs), calendar.py, contracts.py, analytics.py, lookups.py, rentals.py, push.py
Cœur	core/	Base de données, sécurité et jetons, configuration, cache

Le fichier **calc.py** est l'unique source des règles de calcul (net, pourcentage, prorata du carburant, couleurs) : aucune formule n'est dupliquée.

3.4 Modèle de données

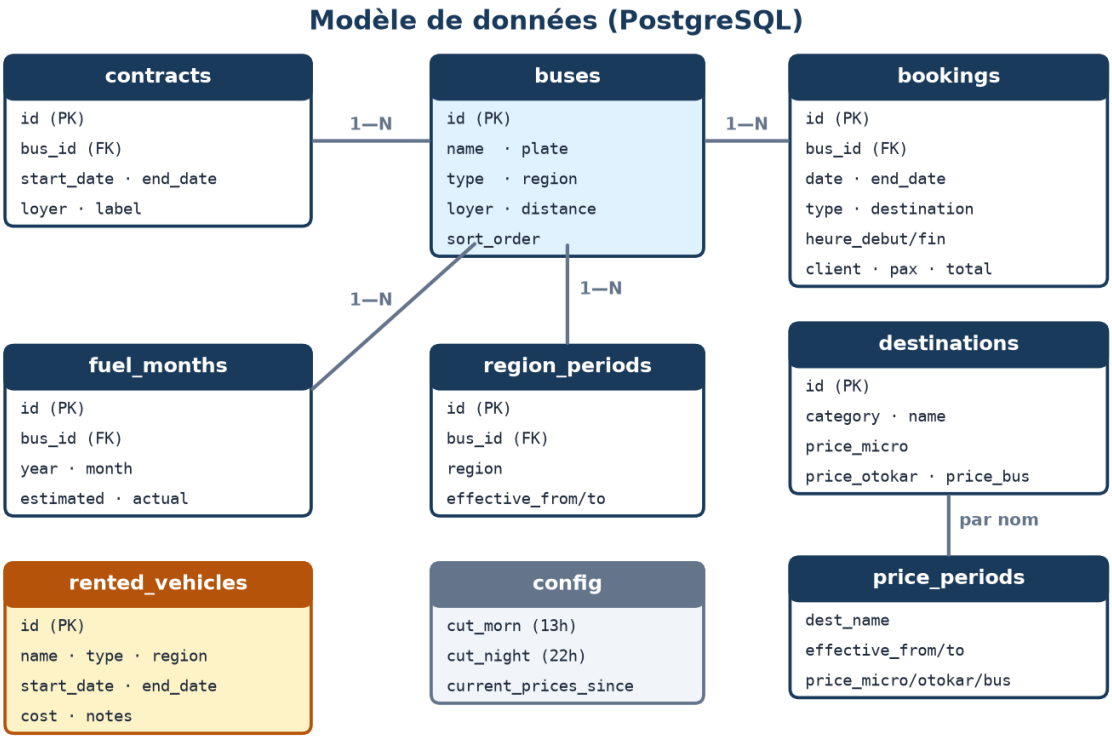


Figure 2 — Modèle de données (simplifié)

Table	Contenu	Remarque
buses	Véhicules : nom, plaque, type, région actuelle, distance, ordre	Le nom est unique
bookings	Sorties et indisponibilités : date, fin, destination, client, passagers, prix, heures	Liée au véhicule par **clé étrangère** (jamais par le nom)
contracts	Contrats : dates libres, loyer, libellé	Plusieurs contrats possibles par véhicule
fuel_months	Carburant par véhicule et par mois : estimé, réel	Unicité véhicule + année + mois
region_periods	Historique des régions : région, du … au …	Les chevauchements sont refusés
destinations / price_periods	Catalogue et prix historiques par type de véhicule	Prix retenu selon la date de la sortie

Table	Contenu	Remarque
rented_vehicles	Locations externes de renfort	Hors flotte : sans lien avec buses
config	Seuils horaires, date d'application des prix	Ligne unique
audit_log, push_subscriptions	Journal des actions ; téléphones abonnés aux notifications	—

Données initiales migrées depuis l'ancien classeur : **23 véhicules**, **944 sorties**, **120 destinations**, ainsi que les contrats et le carburant.

3.5 Interface de programmation (API)

Groupe	Principales routes	Accès
Authentification	POST /api/login · POST /api/logout	Public (connexion) / connecté
Calendrier et jours	GET /api/calendar/{année}/{mois} · GET /api/day/… · POST /api/day · /api/excursions	Lecture : connecté · Écriture : administrateur
Véhicules et régions	/api/buses · /api/buses/overview · /api/region-periods (+ /move)	Lecture : connecté · Écriture : administrateur
Contrats et carburant	/api/contracts · /api/fuel	Lecture : connecté · Écriture : administrateur
Destinations et prix	/api/destinations · /api/price-periods · /api/config	Lecture : connecté · Écriture : administrateur
Locations de renfort	/api/rented-vehicles · /api/rented-vehicles/alternatives	Lecture : connecté · Écriture : administrateur
Analyse	GET /api/analytics/{année}/{mois}	Administrateur
Exports	GET /api/export/… (dix exports Excel)	Connecté / administrateur
Journal	GET /api/audit (complet) · GET /api/audit/changes	Propriétaire / administrateur
Notifications	/api/push/… · POST /api/notify/announce · /api/notify/test	Connecté / administrateur
Supervision	GET /api/health · GET /api/health/db	Public (état du service)

3.6 Sécurité

Mesure	Détail
Authentification	Jeton signé (HMAC-SHA256) contenant l'utilisateur et la date d'expiration ; invalidé automatiquement si le mot de passe change
Contrôle des droits	Vérifié **côté serveur** à chaque requête (lecture seule, administrateur, propriétaire) ; l'interface ne fait que refléter ces droits
Protection contre les essais répétés	6 échecs de connexion par adresse en 5 minutes, puis blocage temporaire ; les échecs sont journalisés
Secrets	Mots de passe, clés et adresse de la base uniquement dans des variables d'environnement, jamais dans le code
Base de données	Sécurité au niveau des lignes (RLS) activée sur toutes les tables ; connexion chiffrée ; requêtes paramétrées via l'ORM
Transport et navigateur	HTTPS partout ; origines autorisées limitées (CORS) ; en-têtes de sécurité (nosniff, anti-framing, pas de référent)

Mesure	Détail
Erreurs	Les erreurs inattendues renvoient un message lisible sans exposer d'adresse, d'identifiant ou de mot de passe
Traçabilité	Journal de chaque ajout, modification, suppression et des connexions (visibles du propriétaire)

> **Amélioration de sécurité prévue**
> Les comptes sont définis dans une variable d'environnement du serveur et les mots de passe n'y sont pas hachés. C'est acceptable pour une petite équipe, mais l'étape suivante est une table d'utilisateurs avec mots de passe hachés et une page de gestion (F24).

3.7 Déploiement et exploitation

Élément	Service	Détail
Interface	Vercel	Construit et publié automatiquement à chaque envoi sur la branche principale ; variable VITE_API_URL
API	Render (région Francfort)	Python 3 ; démarrage : uvicorn app.main:app ; variables : DATABASE_URL, BUS_USERS, BUS_SECRET, CORS_ORIGINS, clés VAPID
Base	Supabase (Stockholm)	PostgreSQL ; connexion par le pooler SSL ; tables créées au démarrage de l'API
Surveillance	UptimeRobot	Appel de /api/health toutes les 5 minutes ; /api/health/db vérifie l'accès à la base
Code	GitHub	Dépôt unique ; chaque envoi déclenche les déploiements

3.8 Performance — mesures

Mesure	Avant	Après
Requêtes SQL pour afficher le calendrier d'un mois	52	**8**
Requêtes SQL pour la page Analyse	50	**6**
Requêtes SQL pour la liste des contrats	76	**2**
Taille de la réponse du calendrier	17 Ko	**1,8 Ko** (compressée)
Latence API → base de données	—	environ 0,1 s (Francfort → Stockholm)
Réponse de l'API en régime chaud	—	environ 0,3 s
Mois déjà consulté	chargement complet	affichage **immédiat** puis actualisation silencieuse

Limite connue : sur l'offre gratuite de l'hébergeur, l'API se met en veille après 15 minutes d'inactivité et met 30 à 60 secondes à redémarrer. La surveillance régulière l'évite ; un abonnement payant la supprime.

4. Interface et ergonomie (mobile d'abord)

4.1 Principes

- **Le pouce d'abord** : navigation en barre d'onglets en bas de l'écran sur téléphone, fiche d'un jour qui glisse depuis le bas, principaux boutons d'au moins 44 px.
- **L'essentiel en premier** : les totaux par région et le net du mois sont affichés au-dessus des véhicules.
- **Aujourd'hui toujours visible** : la colonne du jour est mise en évidence et le tableau s'ouvre positionné dessus.
- **Deux affichages au choix** : « Cartes » (une carte par véhicule, adaptée au téléphone) ou « Tableau » (vue d'ensemble sur grand écran) ; le choix est mémorisé.
- **Tableaux lisibles sur téléphone** : chaque ligne des tableaux (véhicules, destinations, prix, régions, locations, journal) devient une carte étiquetée ; les formulaires s'ouvrent en bas d'écran ; les sous-onglets des paramètres défilent.
- **Retours clairs** : squelettes de chargement, messages d'erreur lisibles, indication « estimé » quand le carburant n'est pas définitif.
- **Accessibilité** : zoom autorisé, étiquettes pour les boutons-icônes, contrastes élevés, champs de saisie en 16 px.
- **Application installable** : invitation à l'installation sur Android, consigne d'ajout à l'écran d'accueil sur iPhone.

4.2 Écrans

Écran	Rôle	Contenu
Connexion	Tous	Identifiant et mot de passe, affichage du mot de passe, message d'erreur clair
Calendrier	Tous	Net du mois, totaux par région, cartes ou tableau des véhicules, pastilles de couleur, export Excel
Fiche d'un jour	Tous (écriture : admin)	Sorties du jour d'un véhicule, ajout / modification, calcul automatique du prix, contrôle de couverture
Contrats	Tous (écriture : admin)	Contrats par véhicule, résultat détaillé, carburant mensuel estimé / réel
Loués	Tous (écriture : admin)	Véhicules de renfort, coût réparti, vérification d'alternative
Analyse	Administrateur	Couverture du loyer, occupation, heures d'activité par véhicule
Paramètres	Tous (écriture : admin)	Véhicules, destinations, prix, régions par saison, seuils horaires
Journal	Administrateur	Historique des modifications ; envoi d'une annonce à tous les téléphones

5. Qualité, tests et recette

5.1 Tests automatiques

24 tests s'exécutent en moins de 3 secondes sur une base temporaire (la vraie base n'est jamais touchée). Commande : **pytest** dans le dossier backend.

Fichier	Tests	Ce qui est vérifié
test_calc.py	6	Carburant au prorata (exemple du § 2.2 au centime près), résultat estimé ou réel, recettes limitées au contrat, pourcentage, location sur deux mois, couleurs des jours
test_regions.py	11	Écriture unique des noms de région, véhicule qui ne bouge pas, découpage exact au jour du déplacement, absence de faux déplacement dû à une faute de frappe
test_calendar.py	4	Un seul bloc par région, une ligne par contrat en cas de changement de région, totaux régionaux = somme des loyers, véhicule sans contrat absent
test_analytics.py	2	La page Analyse découpe le mois exactement comme le calendrier : mêmes lignes, mêmes montants (recette, loyer, carburant, net)
test_cache.py	1	Le cache est vidé dès qu'une écriture a lieu : jamais de donnée périmée

5.2 Cahier de recette (tests manuels)

N°	Scénario	Résultat attendu
R01	Se connecter avec un compte administrateur	Accès au calendrier du mois en cours, six onglets visibles
R02	Saisir un mauvais mot de passe	Message « Identifiants incorrects » ; après 6 essais, blocage temporaire
R03	Ouvrir le calendrier sur téléphone	Vue cartes, totaux en haut, jour d'aujourd'hui entouré, barre d'onglets en bas
R04	Toucher un jour d'un véhicule, ajouter une sortie	Fiche en bas d'écran ; prix calculé ; pastille colorée après enregistrement
R05	Toucher un jour hors contrat	Case grisée, saisie impossible
R06	Créer un contrat et saisir le carburant (exemple du § 2.2)	Résultat 5 613,23 TND, marqué « estimé » tant que juin n'est pas réel
R07	Déplacer un véhicule vers une autre région au milieu du mois	Le véhicule apparaît dans les deux régions, avec ses propres jours
R08	Deux contrats consécutifs dans deux régions	Deux lignes (une par contrat) ; totaux régionaux corrects
R09	Se connecter avec un compte lecture seule	Consultation possible, boutons de modification absents ou refusés
R10	Exporter le calendrier en Excel	Fichier .xlsx identique à l'écran, avec le découpage par région
R11	Consulter le journal après une modification	Ligne avec utilisateur, heure et détail exact
R12	Activer la cloche, puis envoyer une annonce (Journal)	Notification reçue sur le téléphone
R13	Installer l'application, couper le réseau, rouvrir	L'application s'ouvre et affiche les dernières données
R14	Changer de mois avec les flèches	Mois voisins affichés instantanément
R15	Ouvrir Paramètres > Véhicules et Loués sur téléphone	Une carte étiquetée par ligne, sans défilement horizontal ; formulaire en bas d'écran

6. Planning et organisation

Phase	Contenu	État
1 — Analyse	Étude de l'ancien classeur, identification des erreurs, règles de gestion, migration des données (23 véhicules, 944 sorties, 120 destinations)	**Réalisé**
2 — Conception et API	Modèle de données, règles de calcul centralisées, API sécurisée avec rôles et journal	**Réalisé**
3 — Interface	Calendrier, saisie d'un jour, contrats, carburant, paramètres, analyse, exports	**Réalisé**
4 — Mise en ligne	Base Supabase, API Render, interface Vercel, notifications push, application installable	**Réalisé**
5 — Optimisation	Réduction des requêtes, cache, compression, déplacement de l'API à Francfort, robustesse aux réveils du serveur	**Réalisé**
6 — Mobile et finitions	Barre d'onglets, vue cartes, tableaux en cartes, formulaires en bas d'écran, notifications accessibles, invitation à l'installation	**Réalisé**
7 — Recette et documentation	24 tests automatiques, cahier de recette, ce cahier des charges, guide de déploiement	**Réalisé**
8 — Évolutions	Voir § 9	**Prévu**

7. Budget et hébergement

Service	Offre actuelle	Coût mensuel	Limites à connaître
Vercel (interface)	Hobby	0	Usage personnel / non commercial selon les conditions du service
Render (API)	Gratuite	0	Mise en veille après 15 min ; 512 Mo de mémoire
Supabase (base)	Gratuite	0	500 Mo ; projet mis en pause après une semaine sans activité
UptimeRobot	Gratuite	0	Contrôle toutes les 5 minutes
GitHub	Gratuite	0	—
Total		**0 TND**	

Option production (tarifs indicatifs, à vérifier auprès des fournisseurs) : l'offre Starter de Render (environ 7 USD par mois) supprime la mise en veille ; Supabase Pro (environ 25 USD par mois) ajoute les sauvegardes quotidiennes ; un nom de domaine propre coûte environ 12 USD par an.

8. Risques et limites connues

Risque	Probabilité	Impact	Parade
Mise en veille de l'API (démarrage à froid de 30 à 60 s)	Moyenne	Moyen	Surveillance toutes les 5 min ; relances automatiques ; offre payante possible
Projet Supabase gratuit mis en pause après inactivité	Faible	Fort	Usage régulier ; réactivation en un clic ; sauvegarde à automatiser (F25)
Perte de données	Faible	Fort	Script de sauvegarde existant ; sauvegardes automatiques planifiées (F25) ou offre payante

Risque	Probabilité	Impact	Parade
Comptes et mots de passe gérés côté serveur sans hachage	Faible	Moyen	Passage à des comptes en base avec mots de passe hachés (F24)
Mise à jour d'une dépendance qui casse le démarrage (déjà rencontré avec le pilote PostgreSQL)	Moyenne	Moyen	Pilote désormais nommé explicitement ; tests automatiques ; versions à figer
Saisie erronée d'une région, d'un prix ou d'une date	Moyenne	Moyen	Noms de région normalisés, contrôle de couverture, journal des modifications
Absence de tests de bout en bout dans le navigateur	Moyenne	Faible	Tests de l'interface à ajouter ; recette manuelle documentée (§ 5.2)

9. Évolutions prévues

Horizon	Évolution	Bénéfice
Court terme (1 mois)	Sauvegarde automatique quotidienne de la base Figement des versions des dépendances	Cohérence des chiffres ; sécurité des données ; stabilité
Moyen terme (3 mois)	Gestion des utilisateurs dans l'interface, mots de passe hachés Tests automatiques de l'interface (bout en bout) et déploiement contrôlé Tableau de bord de direction (évolution du net sur 12 mois) Saisie plus rapide sur téléphone (suggestions, raccourcis)	Autonomie de l'administration ; qualité ; pilotage ; confort mobile
Long terme (6 mois et plus)	Facturation client et factures PDF Interface en arabe et en français Suivi des kilomètres et de la maintenance (alertes d'entretien) Saisie hors ligne avec synchronisation Suivi GPS des véhicules	Couverture complète du métier ; accessibilité ; exploitation terrain

10. Livrables et conditions de recette

10.1 Livrables

- Application en ligne : https://bus-app-web-lemon.vercel.app
- Code source complet (interface, API, tests) : https://github.com/bilelsdn21/bus-app-web
- Documentation : README (règles de calcul, architecture), guide de déploiement, ce cahier des charges
- Suite de 24 tests automatiques et cahier de recette manuel (§ 5.2)
- Exports Excel équivalents à l'ancien classeur

10.2 Critères d'acceptation

- Les 15 scénarios de recette (§ 5.2) sont validés
- Les 24 tests automatiques passent
- Les totaux du calendrier, de l'export Excel et des cartes régionales concordent
- Un utilisateur en lecture seule ne peut rien modifier, y compris en appelant l'API directement
- Le calendrier d'un mois s'affiche en moins de 3 secondes sur une connexion mobile ordinaire (hors réveil du serveur)

Annexe A — Glossaire

Terme	Définition
Contrat	Période d'exploitation d'un véhicule, aux dates libres, avec un loyer
Loyer	Montant payé pour le véhicule sur la période d'un contrat
Sortie / excursion	Trajet payant d'un véhicule : destination, heures, client, prix
Net	Recettes − loyer − carburant
Prorata	Répartition d'un montant au nombre de jours concernés
Estimé / réel	Carburant prévisionnel, remplacé par le montant réel le mois suivant
Région	Zone d'exploitation (Sousse, Djerba…) ; un véhicule peut en changer à une date donnée
Tronçon	Partie d'un contrat passée dans une seule région
Véhicule loué (renfort)	Véhicule externe loué temporairement, hors flotte
PWA	Application web installable sur l'écran d'accueil, utilisable en plein écran
Notification push	Message affiché sur le téléphone même quand l'application est fermée
RLS	Sécurité au niveau des lignes de la base de données
API	Interface par laquelle l'application échange des données avec le serveur

Annexe B — Guide rapide utilisateur

Premier démarrage sur téléphone

- Ouvrir l'adresse de l'application dans le navigateur, se connecter.
- **Android :** accepter l'invitation « Installer l'application ». **iPhone :** bouton Partager, puis « Sur l'écran d'accueil ».
- Toucher **Activer** dans la carte « Restez informé » pour recevoir les notifications.

Au quotidien

- **Voir le mois :** onglet Calendrier ; flèches pour changer de mois ; bascule Cartes / Tableau.
- **Saisir une sortie :** toucher un jour d'un véhicule ; choisir la destination et les heures ; Enregistrer.
- **Lire une carte :** nom du véhicule, net du mois en couleur, jours colorés selon la période de la journée, jour d'aujourd'hui entouré.
- **Voir un contrat :** toucher le nom du véhicule.
- **Exporter :** bouton « Exporter Excel » en haut du calendrier.

Administrateur

- **Envoyer une annonce :** onglet Journal, rubrique « Envoyer une notification à tous ».
- **Déplacer un véhicule de région :** Paramètres, régions par saison, « Déplacer », avec la date du changement.

Annexe C — Configuration et installation

Variables d'environnement

Variable	Où	Rôle
DATABASE_URL	API	Adresse de la base PostgreSQL ; absente = SQLite local
BUS_USERS	API	Comptes au format utilisateur:motdepasse:rôle, séparés par des virgules
BUS_SECRET	API	Clé de signature des jetons (longue, aléatoire)
CORS_ORIGINS	API	Adresses autorisées à appeler l'API
VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY / VAPID_SUBJECT	API	Clés des notifications push
NOTIFY_SECRET	API	Secret du déclencheur hebdomadaire de notifications
TOKEN_TTL_HOURS	API	Durée de validité des jetons (365 jours par défaut)
VITE_API_URL	Interface	Adresse de l'API (variable normale, pas un secret)

Installation locale

API : dans le dossier backend, créer un environnement virtuel, installer requirements.txt, lancer « python -m app.seed » puis « uvicorn app.main:app --reload --port 8000 ».

Interface : dans le dossier frontend, « npm install » puis « npm run dev » (http://localhost:5173).

Tests : dans le dossier backend, installer requirements-dev.txt puis lancer « pytest ».

Annexe D — Trame de soutenance et scénario de démonstration

Durée conseillée : 10 à 12 minutes

Minutes	Séquence	Message clé
0 – 1	Contexte et problème : l'ancien classeur, l'erreur de 1 440 TND	Un vrai problème métier, mesurable
1 – 2	Objectifs et périmètre	Fiabilité, mobilité, rentabilité, traçabilité
2 – 4	Règles de gestion : prorata du carburant (exemple chiffré), régions datées	La partie difficile est résolue et testée
4 – 5	Architecture : une image, trois services, coût nul	Choix techniques justifiés
5 – 9	**Démonstration en direct** (scénario ci-dessous)	L'outil fonctionne, sur téléphone aussi
9 – 10	Qualité : 24 tests, sécurité, journal, mesures de performance (52 → 8 requêtes)	Un projet fiable et rapide
10 – 11	Limites honnêtes et évolutions	Maîtrise du sujet, vision
11 – 12	Conclusion et questions	Valeur apportée

Scénario de démonstration

1. Sur téléphone : ouvrir l'application, montrer le calendrier en cartes, les totaux par région et le jour d'aujourd'hui entouré.
2. Toucher un jour, ajouter une sortie : le prix se calcule seul, la pastille change de couleur.
3. Ouvrir un contrat : montrer le résultat avec le carburant au prorata et la mention « estimé ».
4. Montrer un véhicule qui change de région en cours de mois : une ligne par région, des totaux justes.
5. Compte en lecture seule : les modifications sont refusées.
6. Journal : la modification qui vient d'être faite apparaît avec son auteur.
7. Envoyer une annonce : la notification arrive sur le téléphone, devant le jury.
8. Exporter en Excel : montrer l'équivalence avec l'ancien classeur.

Checklist avant la présentation

• **Réveiller le serveur 5 minutes avant** en ouvrant l'application (le premier chargement après une longue veille peut durer jusqu'à une minute).
• Vérifier la connexion Internet de la salle ; prévoir le partage de connexion du téléphone.
• Tester les comptes (administrateur et lecture seule) et la notification la veille.
• Préparer des captures d'écran et une courte vidéo de secours.
• Choisir un mois contenant un véhicule qui change de région pour la démonstration.